

Anchor DETR: 基于Transformer的目标检测查询设计

王英明 张翔宇 杨桐 孙剑

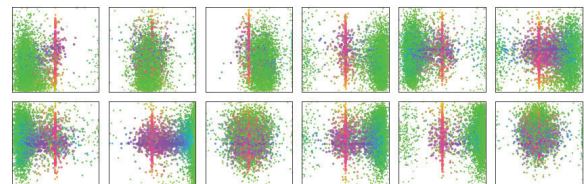
旷视科技 {王英明、张翔宇、杨桐、孙剑}@megvii.com

摘要

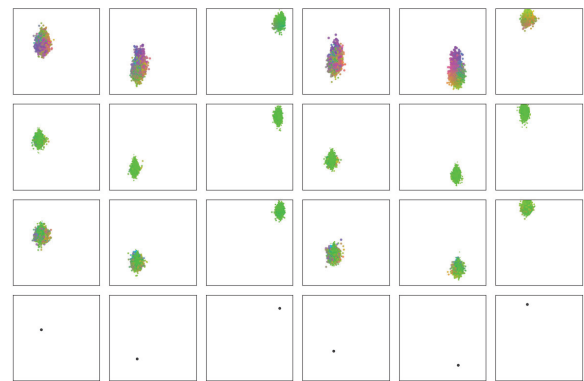
本文提出了一种基于Transformer的目标检测新查询设计。在以往的基于Transformer的检测器中，对象查询是一组学习得到的嵌入向量。然而，每个学习到的嵌入向量没有明确的物理意义，我们无法解释它将关注何处。由于每个对象查询的预测槽没有特定模式，因此难以优化。换言之，每个对象查询不会专注于特定区域。为解决这些问题，在我们的查询设计中，对象查询基于锚点——这一方法在基于CNN的检测器中已被广泛使用。因此，每个对象查询会聚焦于锚点附近的物体。此外，我们的查询设计可以在单一位置预测多个物体，以解决“一个区域，多个物体”的难题。同时，我们设计了一种注意力变体，在达到与DETR中标准注意力相当或更优性能的同时，能够降低内存消耗。得益于查询设计和注意力变体，我们提出的检测器（称为Anchor DETR）在减少10×倍训练周期的情况下，相比DETR实现了更优性能与更快速度。例如，在使用ResNet50-DC5特征训练50个周期时，该方法在MSCOCO数据集上达到44.2 AP与19 FPS。在MSCOCO基准测试上的大量实验证明了所提方法的有效性。代码发布于<https://github.com/megvii-research/AnchorDETR>。

引言

目标检测任务旨在预测图像中每个感兴趣物体的边界框和类别。过去几十年间，基于卷积神经网络的目标检测取得了许多重大进展（Ren et al. 2015; Cai and Vasconcelos 2018; Redmon et al. 2016; Lin et al. 2017; Zhang et al. 2019; Qiao, Chen, and Yuille 2020; Chen et al. 2021）。近期，Carion等人（Carion et al. 2020）提出了基于Transformer架构的全新目标检测范式DETR。该方法通过一组习得的物体查询向量，推理物体间关系及全局图像上下文信息，最终输出预测结果集合。然而，这种习得的物体查询向量难以解释：它缺乏明确的物理意义，且每个物体查询对应的预测槽位不具有特定模式。如图所示



(a) DETR



(b) Ours

图1: 预测槽的可视化。注意子图(a)来自DETR（Carion等人，2020）中的图示。每个预测槽包含验证集上某个查询的所有边界框预测。每个彩色点代表一个预测的归一化中心位置。点按颜色编码：绿色对应小框，红色对应大水平框，蓝色对应大垂直框。子图(b)最后一行的黑点表示锚点。我们的预测槽相比DETR更与特定位置相关联。

图1(a)中，DETR中每个对象查询的预测与不同区域相关联，且每个对象查询将负责一个非常大的区域。这种位置模糊性——即对象查询未聚焦于特定区域——使得优化变得困难。

回顾基于CNN的检测器，锚点与位置高度相关且包含可解释的物理意义。受此启发，我们提出了一种基于锚点的新型查询设计，即我们——

	Feature	AP	GFLOPs	FPS
DETR	DC5	43.3	187	12
SMCA	multi-level	43.7	152	10
Deformable DETR	multi-level	43.8	173	15
Conditional DETR	DC5	43.8	195	10
Ours	DC5	44.2	172	19

表1：与Transformer检测器的比较。结果基于ResNet-50骨干网络，“DC5”表示扩张的C5特征。DETR训练了500个周期，而其他模型训练了50个周期。我们按照DETR的脚本评估所提出检测器的FPS，并将批量大小设为1以公平比较。请注意，我们遵循DETR的脚本编写模型以评估速度。

将锚点编码为对象查询。对象查询是锚点坐标的编码，因此每个对象查询都具有明确的物理意义。然而，这种方案会遇到困难：同一位置可能出现多个对象。在这种情况下，该位置仅有一个对象查询无法预测多个对象，因此需要其他位置的对象查询协同预测这些对象。这将导致每个对象查询负责更大的区域。因此，我们通过为每个锚点添加多种模式来改进对象查询设计，使每个锚点能够预测多个对象。如图1(b)所示，每个对象查询的三种模式的所有预测都分布在对应锚点周围。换言之，这表明每个对象查询仅关注对应锚点附近的物体。因此，所提出的对象查询易于解释。由于对象查询具有特定模式且无需预测远离对应位置的对象，其优化过程也更简便。

除了查询设计之外，我们还设计了一种注意力变体，称为行列解耦注意力（RCDA）。它将二维关键特征解耦为一维行特征和一维列特征，然后依次进行行注意力和列注意力计算。RCDA能够在保持与DETR中标准注意力相当或更优性能的同时，降低内存开销。我们相信它可以作为DETR中标准注意力的有效替代方案。

如表1所示，得益于基于锚点的新型查询设计和注意力变体，所提出的检测器Anchor DETR在使用相同单层特征时，即使训练周期减少10×，也能比原始DETR实现更优的性能和更快的运行速度。与其他训练周期减少10×的类DETR检测器相比，所提出的检测器在其中取得了最佳性能。当使用单层ResNet50-DC5特征（He等人，2016）训练50个周期时，该检测器可实现44.2 AP与19 FPS的性能表现。

主要贡献可概括为：

- 我们提出了一种基于锚点的新型查询设计，适用于基于Transformer的检测器。此外，我们为每个锚点附加多种模式，使其能够预——

为应对“一个区域，多个物体”的难题，我们为每个位置预设了多个对象字典。基于锚点提出的查询方案比学习嵌入更具可解释性且更易优化。得益于所设计的查询机制的有效性，我们的检测器仅用比DETR少10×的训练周期即可获得更优性能。

- 我们设计了一种注意力变体，称为行列解耦注意力。它能在保持与DETR中标准注意力相似甚至更优性能的同时降低内存成本，可作为标准注意力的良好替代方案。
- 我们进行了大量实验，以证明每个组件的有效性。

相对工作

目标检测中的锚点

基于CNN的目标检测器使用两种类型的锚点，即锚框（Ren等人，2015；Lin等人，2017）和锚点（Tian等人，2019；Zhou、Wang和Krähenbühl，2019）。由于手工设计的锚框需要精细调整才能获得良好性能，我们可能更倾向于不使用锚框。通常我们将无锚框视为无需锚框，因此使用锚点的检测器也被视为无锚框方法（Tian等人，2019；Zhou、Wang和Krähenbühl，2019）。DETR（Carion等人，2020）既不采用锚框也不采用锚点，它直接预测图像中每个目标的绝对位置。然而，我们发现将锚点引入目标查询中可能效果更佳。

Transformer检测器

Vaswani等人（Vaswani et al. 2017）首次提出了用于序列转换的transformer模型。最近，Carion等人（Carion et al. 2020）提出了基于transformer的目标检测模型DETR。该transformer检测器会根据查询与键的相似度，将值{v*}的信息传递给查询。Zhu等人（Zhu et al. 2020）提出了Deformable DETR，通过为查询采样可变形点作为值{v*}，并利用多尺度特征来解决transformer检测器收敛速度慢的问题。Gao等人（Gao et al. 2021）则在原始注意力机制中为每个查询添加了高斯映射。

与我们同时，Conditional DETR（Meng等人，2021年）将参考点编码为查询位置嵌入。但动机不同，因此它仅使用参考点生成位置嵌入作为交叉注意力中的条件空间嵌入，而对对象查询仍然是学习到的嵌入。此外，它不涉及一个位置的多次预测和注意力变体。

高效注意力

Transformer的自注意力机制具有较高的复杂度，因此无法很好地处理大量查询和键。为解决这一问题，研究者们提出了多种高效注意力模块（Wang et al. 2020b；

Shen等人2021; Vaswani等人2017; Beltagy、Peters和Cohan 2020; Liu等人2021; Ma等人2021)。一种方法是先计算键和值，这可以使查询或键的数量达到线性复杂度。高效注意力（Shen等人2021）和线性注意力（Katharopoulos等人2020）遵循了这一思路。另一种方法是限制每个查询的键的注意力区域，而不是整个区域。受限自注意力（Vaswani等人2017）、可变形注意力（Zhu等人2020）、十字交叉注意力（Huang等人2019）以及LongFormer（Beltagy、Peters和Cohan 2020）遵循了这一思路。在本文中，我们通过一维全局平均池化将键特征解耦为行特征和列特征，然后依次执行行注意力和列注意力。

方法

锚点

在基于CNN的检测器中，锚点始终是特征图的对应位置。但在基于Transformer的检测器中，它可以更加灵活。锚点可以是学习得到的点、均匀网格点或其他手工设计的锚点。我们采用两种类型的锚点：一种是网格锚点，另一种是学习锚点。如图2(a)所示，网格锚点固定为图像中的均匀网格点。学习锚点则采用0到1的均匀分布随机初始化，并作为学习参数进行更新。通过锚点，预测边界框的中心位置（ $\{v^*\}$ ）将添加到相应的锚点上，作为最终预测，类似于可变形DETR（Zhu等人，2020）中的做法。

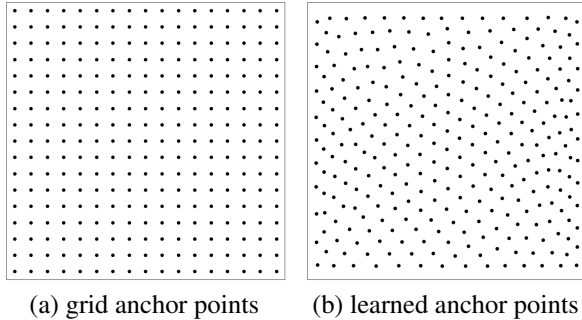


图2: 锚点分布的可视化。每个点代表一个锚点的归一化位置。

注意力机制公式化

类似DETR的transformer注意力机制可表述为:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (1)$$

$$Q = Q_f + Q_p, K = K_f + K_p, V = V_f,$$

其中 d_k 是通道维度，下标 f 表示特征，下标 p 表示位置

嵌入。Q、K、V分别代表查询、键和值。请注意，Q、K、V将分别通过一个线性层，为清晰起见，该部分在公式(1)中省略。

DETR解码器中有两种注意力机制，一种是自注意力，另一种是交叉注意力。在自注意力中， K_f 和 V_f 与 Q_f 相同，而 K_p 与 Q_p 相同。 $Q_f \in \mathbb{R}^{N_q \times C}$ 是上一个解码器的输出，首个解码器的初始 $Q_f^{init} \in \mathbb{R}^{N_q \times C}$ 可设置为常数向量或习得的嵌入。对于查询位置嵌入 $Q_p \in \mathbb{R}^{N_q \times C}$ ，DETR采用一组习得的嵌入:

$$Q_p = \text{Embedding}(N_q, C). \quad (2)$$

在交叉注意力中， $Q_f \in \mathbb{R}^{N_q \times C}$ 由前一层自注意力的输出生成，而 $K_f \in \mathbb{R}^{H \times W \times C}$ 和 $V_f \in \mathbb{R}^{H \times W \times C}$ 则是编码器的输出特征。 $K_p \in \mathbb{R}^{H \times W \times C}$ 是 K_f 的位置嵌入，它通过正弦-余弦位置编码函数（Vaswani et al. 2017; Carion et al. 2020） g_{sin} 基于对应的关键特征位置 $Pos_k \in \mathbb{R}^{H \times W \times 2}$ 生成:

$$K_p = g_{sin}(Pos_k). \quad (3)$$

请注意， H, W, C 表示特征的高度、宽度和通道数，而 N_q 是查询的预定义数量。

锚点到对象查询

通常，解码器中的 Q_p 被视为对象查询，因为它负责区分不同的对象。如公式(2)所示的已学习嵌入的对象查询，正如引言部分所讨论的那样，难以解释。

本文提出基于锚点 Pos_q 设计对象查询。 $Pos_q \in \mathbb{R}^{N_A \times 2}$ 表示 N_A 个点及其 (x, y) 位置（取值范围为0到1）。基于锚点的对象查询可表述为:

$$Q_p = \text{Encode}(Pos_q). \quad (4)$$

这意味着我们将锚点编码为对象查询。

那么如何设计编码函数呢？既然对象查询被设计为如公式(1)所示的查询位置嵌入，最自然的方式就是采用与键相同的位置编码函数:

$$Q_p = g(Pos_q), K_p = g(Pos_k), \quad (5)$$

其中 g 是位置编码函数。位置编码函数可以是 g_{sin} 或其他位置编码函数。在本文中，我们倾向于使用一个包含两个线性层的小型MLP网络进行额外适配，而非仅采用启发式的 g_{sin} 。

每个锚点进行多重预测

为了处理一个位置可能存在多个对象的情况，我们进一步改进了对象查询机制，使每个锚点能够预测多个对象，而非仅单一预测。回顾初始查询特征 $Q_f^{init} \in \mathbb{R}^{N_q \times C}$ ，其中 N_q 个对象查询各自拥有一种模式 $Q_f^i \in \mathbb{R}^{1 \times C}$ 。需注意 i 为对象查询的索引。为预测

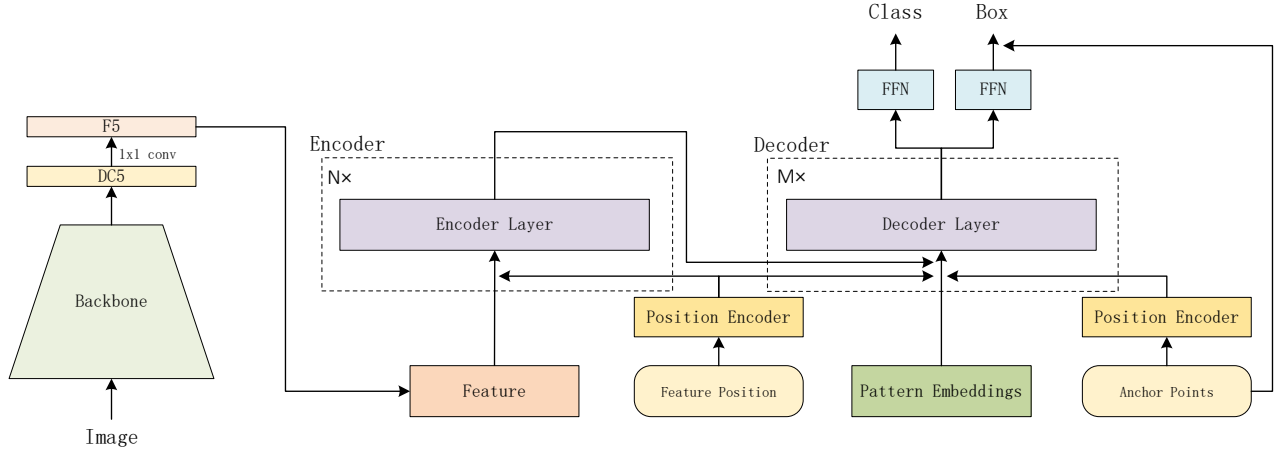


图3: 所提出检测器的流程。注意，编码器层和解码器层与DETR结构相同，只是我们将编码器层中的自注意力机制和解码器层中的交叉注意力机制替换为提出的行列解耦注意力机制。

对于每个锚点的多个对象，我们可以将多种模式融入每个对象查询中。我们使用一小部分模式嵌入 $Q_f^i \in \mathbb{R}^{N_p \times C}$:

$$Q_f^i = \text{Embedding}(N_p, C), \quad (6)$$

为了在每个位置检测具有不同模式的对象。 N_p 是模式的数量，这个数量非常小，例如 $N_p = 3$ 。由于平移不变性的特性，这些模式在所有对象查询之间是共享的。因此，我们可以通过将 $Q_f^i \in \mathbb{R}^{N_p \times C}$ 共享给每个 $Q_p \in \mathbb{R}^{N_A \times C}$ 来获得初始的 $Q_f^{init} \in \mathbb{R}^{N_p N_A \times C}$ 和 $Q_p \in \mathbb{R}^{N_p N_A \times C}$ 。这里的 N_q 等于 $N_p \times N_A$ 。然后，我们可以将提出的模式-位置查询设计定义为：

$$Q = Q_f^{init} + Q_p. \quad (7)$$

对于以下解码器， Q_f 同样像 DETR 那样由上一个解码器的输出生成。

得益于所提出的查询设计，该检测器拥有可解释的查询，并且比原始DETR在训练周期减少10×的情况下实现了更优的性能。

行列解耦注意力

Transformer会消耗大量GPU内存，这可能限制其使用高分辨率特征或其他扩展功能。可变形Transformer (Zhu 等人, 2020) 能够降低内存开销，但会导致内存的随机访问，这可能对硬件不够友好。此外也存在一些具有线性复杂度的注意力模块 (Ma等人, 2021; Shen等人, 2021)，它们不会引发内存随机访问。然而在我们的实验中，我们发现这些注意力模块难以很好地处理类DETR检测器。这可能是因为类DETR解码器中的交叉注意力比自注意力机制更为复杂。

本文提出行-列解耦注意力机制 (RCDA)，它不仅能降低内存负担，还能实现相当甚至更优的性能。

比DETR中的标准注意力更优。RCDA的主要思想是通过一维全局平均池化将关键特征 $K_f \in \mathbb{R}^{H \times W \times C}$ 解耦为行特征 $K_{f,x} \in \mathbb{R}^{W \times C}$ 和列特征 $K_{f,y} \in \mathbb{R}^{H \times C}$ ，随后依次执行行注意力与列注意力计算。在不失一般性的前提下，我们假设 $W \geq H$ ，此时RCDA可表述为：

$$\begin{aligned} A_x &= \text{softmax}\left(\frac{Q_x K_x^T}{\sqrt{d_k}}\right), A_x \in \mathbb{R}^{N_q \times W}, \\ Z &= \text{weighted_sumW}(A_x, V), Z \in \mathbb{R}^{N_q \times H \times C}, \\ A_y &= \text{softmax}\left(\frac{Q_y K_y^T}{\sqrt{d_k}}\right), A_y \in \mathbb{R}^{N_q \times H}, \\ Out &= \text{weighted_sumH}(A_y, Z), Out \in \mathbb{R}^{N_q \times C}, \end{aligned} \quad (8)$$

其中

$$\begin{aligned} Q_x &= Q_f + Q_{p,x}, \quad Q_y = Q_f + Q_{p,y}, \\ Q_{p,x} &= g_{1D}(Pos_{q,x}), \quad Q_{p,y} = g_{1D}(Pos_{q,y}), \\ K_x &= K_{f,x} + K_{p,x}, \quad K_y = K_{f,y} + K_{p,y}, \\ K_{p,x} &= g_{1D}(Pos_{k,x}), \quad K_{p,y} = g_{1D}(Pos_{k,y}), \\ V &= V_f, \quad V \in \mathbb{R}^{H \times W \times C}. \end{aligned} \quad (9)$$

加权求和W和加权求和H操作分别沿宽度维度和高度维度进行加权求和。 $Pos_{q,x} \in \mathbb{R}^{N_q \times 1}$ 是 $Q_f \in \mathbb{R}^{N_q \times C}$ 对应的行位置，而 $Pos_{q,y} \in \mathbb{R}^{N_q \times 1}$ 、 $Pos_{k,x} \in \mathbb{R}^{W \times 1}$ 、 $Pos_{k,y} \in \mathbb{R}^{H \times 1}$ 的情况类似。 g_{1D} 是一维位置编码函数，类似于 g ，它将一维坐标编码为具有C个通道的向量。

现在我们来分析为何它能节省内存。为了清晰起见，我们在之前的表述中将多头注意力的头数设为1，但这并不失一般性；然而在内存分析中，我们需要考虑头数。DETR中注意力的主要内存开销

	Anchor-Free	NMS-Free	RAM-Free	Epochs	AP	AP_{50}	AP_{75}	AP_s	AP_m	AP_l
RetinaNet			✓	36	38.7	58.0	41.5	23.3	42.3	50.3
FCOS	✓		✓	36	41.4	60.1	44.9	25.6	44.9	53.1
POTO	✓	✓	✓	36	41.4	59.5	45.6	26.1	44.9	52.0
Faster RCNN				36	40.2	61.0	43.8	24.2	43.5	52.0
Cascade RCNN				36	44.3	62.2	48.0	26.6	47.7	57.7
Sparse RCNN	✓	✓		36	44.5	63.4	48.2	26.9	47.2	59.5
DETR-C5	✓	✓	✓	500	42.0	62.4	44.2	20.5	45.8	61.1
DETR-DC5	✓	✓	✓	500	43.3	63.1	45.9	22.5	47.3	61.1
SMCA	✓	✓	✓	50	43.7	63.6	47.2	24.2	47.0	60.4
Deformable DETR	✓	✓		50	43.8	62.6	47.7	26.4	47.1	58.0
Ours-C5	✓	✓	✓	50	42.1	63.1	44.9	22.3	46.2	60.0
Ours-DC5	✓	✓	✓	50	44.2	64.7	47.5	24.7	48.2	60.6
DETR-C5-R101	✓	✓	✓	500	43.5	63.8	46.4	21.9	48.0	61.8
DETR-DC5-R101	✓	✓	✓	500	44.9	64.7	47.7	23.7	49.5	62.3
Ours-C5-R101	✓	✓	✓	50	43.5	64.3	46.6	23.2	47.7	61.4
Ours-DC5-R101	✓	✓	✓	50	45.1	65.7	48.8	25.8	49.4	61.6

表2：与其他检测器的比较。结果基于ResNet-50骨干网络（若无特殊说明），R101表示ResNet-101骨干网络。“C5”、“DC5”分别表示使用单一C5或DC5特征的检测器。其他未特殊说明的检测器均使用多层级特征。RAM指内存随机访问，在实际硬件中通常不友好。

RCDA	anchors	patterns	AP	AP_{50}	AP_{75}	AP_s	AP_m	AP_l
			39.3	59.4	41.8	20.7	42.6	55.0
	✓	✓	44.2	65.3	47.2	24.4	47.8	61.8
✓			40.3	61.6	42.5	21.7	44.1	56.3
✓		✓	40.3	60.8	43.0	21.1	44.2	57.0
✓	✓		42.6	63.6	45.5	23.2	46.4	58.3
✓	✓	✓	44.2	64.7	47.5	24.7	48.2	60.6

表3：各组成部分的有效性。RCDA表示用RCDA替换标准注意力机制。“锚点”指使用锚点生成对象查询。“模式”表示为每个对象查询分配多个模式，从而使每个位置具有多个预测结果。

是注意力权重图 $A \in \mathbb{R}^{N_q \times H \times W \times M}$ 。但RCDA的注意力权重图是 $A_x \in \mathbb{R}^{N_q \times W \times M}$ 和 $A_y \in \mathbb{R}^{N_q \times H \times M}$ ，其内存消耗远小于 A 。然而，RCDA中主要的内存消耗来自临时结果 Z 。因此我们应比较 $A \in \mathbb{R}^{N_q \times H \times W \times M}$ 和 $Z \in \mathbb{R}^{N_q \times H \times C}$ 的内存消耗。RCDA节省内存的比率为：

$$r = (N_q \times H \times W \times M) / (N_q \times H \times C) = W \times M / C \quad (10)$$

默认设置为 $M = 8, C = 256$ 。因此当较大侧 W 等于32时（这是目标检测中C5特征的典型值），内存成本大致相同。使用高分辨率特征时能够节省内存，例如对于C4或DC5特征可节省约2倍内存，对于C3特征则可节省约4倍内存。

实验

实现细节

我们在MS COCO (Lin等人, 2014) 基准测试上进行了实验。所有模型均在train2017数据集上训练。

在val2017数据集上进行分割和评估。模型使用8个GPU进行训练，每个GPU处理1张图像。我们在训练集上使用AdamW优化器 (Loshchilov和Hutter, 2017) 训练模型50个周期，其中骨干网络的初始学习率设为 10^{-5} ，其他部分设为 10^{-4} 。学习率将在第40个周期时衰减为原来的0.1倍。权重衰减设置为 10^{-4} ，丢弃率设为0。注意力头数为8，注意力特征通道数为256，前馈网络的隐藏维度为1024。默认情况下，我们选择锚点数量为300，模式数量为3。默认使用一组学习得到的点作为锚点。编码器和解码器层数均为6，与DETR相同。分类损失采用焦点损失 (Lin等人, 2017)，遵循可变形DETR的方法。

主要结果

如表2所示，我们将提出的检测器与RetinaNet (Lin等人, 2017)、FCOS (Tian等人, 2019)、POTO (Wang等人, 2020a)、Faster RCNN (Ren等人, 2015)、Cascaded RCNN (Cai和Vasconcelos, 2018)、Sparse RCNN (Sun等人, 2020)、DETR (Carion等人, 2020) 进行了比较。

anchor points	patterns	AP	AP_{50}	AP_{75}	AP_s	AP_m	AP_l
100	1	40.8	61.5	43.6	21.0	44.8	57.5
100	3	43.4	63.7	46.5	23.8	47.1	60.9
300	1	42.6	63.6	45.5	23.2	46.4	58.3
300	3	44.2	64.7	47.5	24.7	48.2	60.6
900	1	42.9	63.8	46.1	24.4	46.7	58.9

表4: 单点位置多重预测对比。我们展示了不同锚点数量和模式下的性能表现。请注意当模式数量为1时, 该方法会退化为单锚点的单一预测。

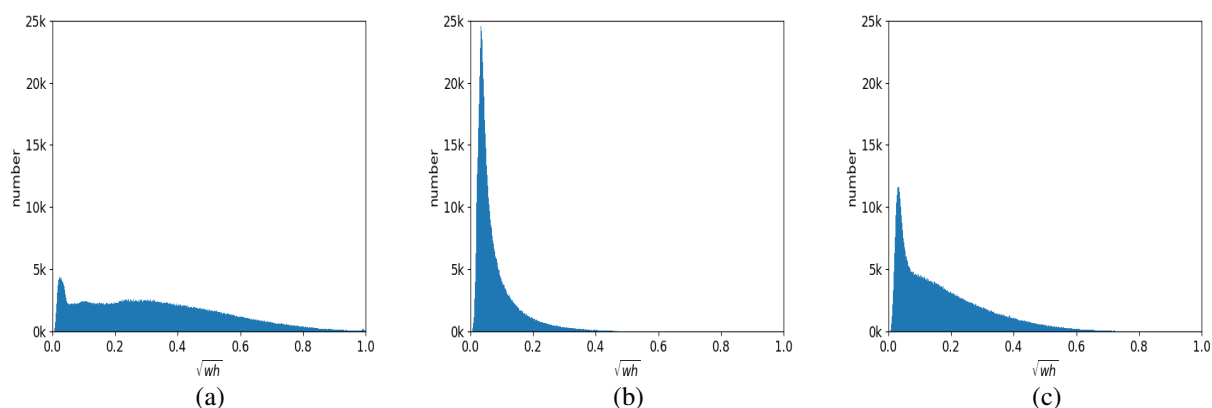


图4: 各模式的直方图。我们展示了不同模式下预测框尺寸的直方图。横坐标表示预测框面积的平方根, 该预测框的宽度“w”和高度“h”已归一化。大尺寸框通常出现在模式 (a) 中, 模式 (b) 主要关注小物体, 而模式 (c) 介于两者之间。

	AP	AP_{50}	AP_{75}	AP_s	AP_m	AP_l
grid	44.1	64.5	47.6	24.8	48.0	60.9
learned	44.2	64.7	47.5	24.7	48.2	60.6

表5: 不同类型锚点的比较。“learned”表示使用300个学习得到的锚点。“grid”表示使用手工设计的网格锚点, 数量为 17×17 , 接近300。

SMCA (Gao等人, 2021年) 和Deformable DETR (Zhu等人, 2020年)。“C5”和“DC5”表示检测器使用单个C5或扩张C5特征, 而其他检测器使用多级特征。使用多级特征通常能获得更好的性能, 但会消耗更多资源。令人惊讶的是, 我们采用单个DC5特征的检测器能够实现比使用多级特征的Deformable DETR和SMCA更优的性能。所提出的检测器在训练周期减少 $10 \times$ 的情况下, 性能仍优于DETR。这证明了所提出的查询设计非常有效。

我们还在表2中展示了每个检测器的无锚框、无NMS和无RAM特性。无锚框和无NMS表示检测器不需要手工设计的锚框和非极大值抑制。无RAM意味着检测器不会涉及任何随机内存访问, 这在实践中对硬件非常友好。

两阶段检测器通常无法实现RAM无访问, 因为感兴趣区域 (RoI) 对硬件而言是随机的, 且RoI-Align (He等人, 2017) /RoI-Pooling (Girshick, 2015) 操作会涉及内存的随机访问。可变形DETR与两阶段检测器类似, 其采样点位置对硬件具有随机性, 因此同样无法实现RAM无访问。相反, 本文提出的检测器继承了DETR的无锚框、无NMS及RAM无访问的特性, 同时在性能上有所提升, 且所需训练周期更短。

消融研究

各组件有效性 表3展示了我们提出的各组件有效性。基于锚点的查询设计能够为每个位置预测多个对象, 将性能从39.3 AP提升至44.2 AP。4.9 AP的改进证明, 具有特定聚焦区域的查询设计更易于优化。在注意力变体RCDA方面, 采用所提查询设计时其性能与标准注意力机制相当, 这证实了RCDA能在降低内存成本的同时保持性能不衰减。此外, 采用与DETR相同的原始查询设计时, RCDA可实现1.0 AP的性能提升。我们认为由于注意力图小于标准注意力机制, 其优化难度略有降低。当采用所提查询设计时, 这一增益将会消失。

Attention	Feature	AP	AP_{50}	AP_{75}	AP_s	AP_m	AP_l	memory
Luna (Ma et al. 2021)	DC5	36.2	58.7	37.1	15.4	39.2	54.7	2.9G
Efficient-att (Shen et al. 2021)		34.1	56.8	34.4	13.9	36.0	53.9	2.2G
std-att		44.2	65.3	47.2	24.4	47.8	61.8	10.5G
RCDA		44.2	64.7	47.5	24.7	48.2	60.6	4.4G
std-att	C5	42.2	63.5	44.9	21.9	45.9	61.0	2.7G
RCDA		42.1	63.1	44.9	22.3	46.2	60.0	2.3G

表6: 不同注意力模块的对比。"std-att"表示DETR中的标准注意力模块。线性复杂度注意力模块能显著降低内存占用, 但性能会下降。提出的RCDA在节省内存的同时, 能达到与DETR标准注意力模块相近的性能。

符号具有相同的效果, 使其更易于优化。基于锚点进行单次预测的目标查询带来了2.3 AP的提升, 而每个锚点进行多次预测可进一步获得1.6的改进。将多次预测应用于DETR中的原始目标查询并不会提升性能。这是因为原始目标查询与位置关联性不强, 因此无法从“一个区域, 多次预测”中获益。

每个锚点的多重预测 显示, 每个锚点的多重预测在查询设计中起着至关重要的作用。对于单一预测(1种模式), 我们发现100个锚点效果不够理想, 而900个锚点与300个锚点的性能相近, 因此默认采用300个锚点。多重预测(3种模式)相较于单一预测(1种模式), 在100个和300个锚点下分别实现了2.6和1.6 AP的性能提升。在预测总数相同的情况下, 每个锚点的多重预测(300个锚点, 3种模式)比单一预测(900个锚点, 1种模式)高出1.3 AP。这表明多重预测带来的改进并非源于预测数量的增加。这些结果证明了每个锚点采用多重预测的有效性。

锚点类型 我们尝试了两种类型的锚点, 即网格锚点和学习锚点。如图2所示, 学习锚点在图像中均匀分布, 这与网格锚点相似。我们推测这是因为在大型COCO数据集中, 物体遍布各个位置。我们还发现, 在表4中, 网格锚点能够达到与学习锚点相似的性能。

目标查询的预测槽位如图1(b)所示, 我们可以观察到, 所提出检测器中每个目标查询的预测槽位会聚焦于对应锚点附近的物体。由于锚点存在三种模式, 我们在图4中展示了每种模式的直方图分布。我们发现这些模式与物体尺寸相关: 大尺寸边界框通常出现在模式(a)中, 而模式(b)主要关注小物体。但查询模式并非仅取决于物体尺寸, 因为小尺寸边界框同样可能出现在模式(a)中。我们认为这是由于小物体数量更多,

并且多个小物体更可能出现在一个区域中。因此, 所有模式都将负责小物体。

行-列解耦注意力 如表6所示, 我们将行-列解耦注意力与DETR中的标准注意力以及一些具有线性复杂度的高效注意力模块进行比较。与标准注意力模块相比, 具有线性复杂度的注意力模块能显著降低训练内存占用。然而, 它们的性能相比标准注意力模块下降了约10 AP。这些高效注意力似乎并不适用于类DETR检测器。相反, 行-列解耦注意力实现了相近的性能。如前所述, 使用高分辨率特征时, 行-列解耦注意力能显著降低内存占用; 而使用C5特征时, 其内存成本与标准注意力大致相同。例如, 在使用DC5特征时, RCDA将内存从10.5G降低至4.4G, 同时获得与标准注意力相同的性能。综上所述, 行-列解耦注意力在保持竞争力的性能同时具有内存高效性, 可成为DETR中标准注意力的理想替代方案。

结论

本文提出了一种基于Transformer的检测器。我们设计了一种基于锚点的新型查询机制, 该锚点具有明确的物理意义。相应的预测槽位可采用特定模式, 即预测结果靠近锚点区域, 从而更易于优化。此外, 我们为每个锚点引入多模式机制, 以解决“单区域多目标”的检测难题。同时, 我们提出了一种行列解耦注意力变体, 该机制在保持与DETR标准注意力相当或更优性能的同时, 能显著降低内存开销。所提出的检测器在比DETR少10×训练轮次的情况下, 实现了更优性能与更快推理速度, 且具备无锚框、无NMS、无RAM依赖的端到端检测特性。我们希望该检测器能应用于实际场景, 并为相关研究提供简洁而强大的基准模型。

致谢

本论文得到科技部国家重点研发计划（项目编号：2020 AAA0104400）和北京智源人工智能研究院（BAAI）的支持。

参考文献

- Beltagy, I.; Peters, M. E.; and Cohan, A. 2020. Longformer : 长文档 Transformer. *arXiv preprint arXiv:2004.05150*.
- Cai, Z.; and Vasconcelos, N. 2018. Cascade R-CNN: 深入高质量目标检测。于 *Proceedings of the IEEE conference on computer vision and pattern recognition*, 6154–6162.
- Carion, N.; Massa, F.; Synnaeve, G.; Usunier, N.; Kirillov, A.; and Zagoruyko, S. 2020. 基于 Transformer 的端到端目标检测。于 *European Conference on Computer Vision*, 213–229.
- Springer. Chen, Q.; Wang, Y.; Yang, T.; Zhang, X.; Cheng, J.; and Sun, J. 2021. 你只需看一层特征。于 *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 13039–13048.
- Gao, P.; Zheng, M.; Wang, X.; Dai, J.; and Li, H. 2021. 通过空间调制协同注意力实现 DETR 快速收敛。 *arXiv preprint arXiv:2101.07448*.
- Girshick, R. 2015. Fast R-CNN. 于 *Proceedings of the IEEE international conference on computer vision*, 1440–1448.
- He, K.; Gkioxari, G.; Dollár, P.; and Girshick, R. 2017. Mask R-CNN. 于 *Proceedings of the IEEE international conference on computer vision*, 2961–2969.
- He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016. 用于图像识别的深度残差学习。于 *Proceedings of the IEEE conference on computer vision and pattern recognition*, 770–778.
- Huang, Z.; Wang, X.; Huang, L.; Huang, C.; Wei, Y.; and Liu, W. 2019. CCNet: 语义分割的交叉注意力。于 *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 603–612.
- Katharopoulos, A.; Vyas, A.; Pappas, N.; and Fleuret, F. 2020. Transformer 即 RNN: 具有线性注意力的快速自回归 Transformer. *arXiv:2006.16236*.
- Lin, T.-Y.; Goyal, P.; Girshick, R.; He, K.; and Dollár, P. 2017. 用于密集目标检测的焦点损失。于 *Proceedings of the IEEE international conference on computer vision*, 2980–2988.
- Lin, T.-Y.; Maire, M.; Belongie, S.; Hays, J.; Perona, P.; Ramanan, D.; Dollár, P.; and Zitnick, C. L. 2014. Microsoft COCO: 上下文中的常见物体。于 *European conference on computer vision*, 740–755.
- Springer. Liu, Z.; Lin, Y.; Cao, Y.; Hu, H.; Wei, Y.; Zhang, Z.; Lin, S.; and Guo, B. 2021. Swin Transformer: 使用移位窗口的分层视觉 Transformer. *arXiv preprint arXiv:2103.14030*.
- Loshchilov, I.; and Hutter, F. 2017. 解耦权重衰减正则化。 *arXiv preprint arXiv:1711.05101*.
- 马, X.; 孔, X.; 王, S.; 周, C.; 梅, J.; 马, H.; 和泽特勒迈尔, L. 2021. Luna: 线性统一嵌套注意力。 *arXiv:2106.01540*.
- 孟, D.; 陈, X.; 范, Z.; 曾, G.; 李, H.; 袁, Y.; 孙, L.; 和王, J. 2021. 用于快速训练收敛的条件 DETR. *arXiv:2108.06152*.
- 乔, S.; 陈, L.-C.; 和尤伊尔, A. 2020. DetectoRS: 通过递归特征金字塔和可切换空洞卷积检测对象。 *arXiv preprint arXiv:2006.02334*.
- 雷德蒙, J.; 迪瓦拉, S.; 吉尔希克, R.; 和法尔哈迪, A. 2016. 你只看一次: 统一、实时的目标检测。于 *Proceedings of the IEEE conference on computer vision and pattern recognition*, 779–788.
- 任, S.; 何, K.; 吉尔希克, R.; 和孙, J. 2015. 更快的 R-CNN: 通过区域提议网络实现实时目标检测。 *arXiv preprint arXiv:1506.01497*.
- 沈, Z.; 张, M.; 赵, H.; 易, S.; 和李, H. 2021. 高效注意力: 具有线性复杂度的注意力。于 *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 3531–3539.
- 孙, P.; 张, R.; 姜, Y.; 孔, T.; 徐, C.; 詹, W.; 富永, M.; 李, L.; 袁, Z.; 王, C.; 等. 2020. 稀疏 R-CNN: 使用可学习提议的端到端目标检测。 *arXiv preprint arXiv:2011.12450*.
- 田, Z.; 沈, C.; 陈, H.; 和何, T. 2019. FCOS: 全卷积单阶段目标检测。于 *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 9627–9636.
- 瓦斯瓦尼, A.; 沙泽尔, N.; 帕尔马, N.; 乌斯佐雷特, J.; 琼斯, L.; 戈麦斯, A. N.; 凯泽, L.; 和波洛苏欣, I. 2017. 注意力就是你所需要的一切。 *arXiv preprint arXiv:1706.03762*.
- 王, J.; 宋, L.; 李, Z.; 孙, H.; 孙, J.; 和郑, N. 2020a. 使用全卷积网络的端到端目标检测。 *arXiv preprint arXiv:2012.03544*.
- 王, S.; 李, B.; 卡布萨, M.; 方, H.; 和马, H. 2020b. Linformer: 具有线性复杂度的自注意力。 *arXiv preprint arXiv:2006.04768*.
- 张, S.; 迟, C.; 姚, Y.; 雷, Z.; 和李, S. Z. 2019. 通过自适应训练样本选择弥合基于锚点与无锚点检测之间的差距。 *arXiv preprint arXiv:1912.02424*.
- 周, X.; 王, D.; 和克亨布尔, P. 2019. 对象即点。于 *arXiv preprint arXiv:1904.07850*.
- 朱, X.; 苏, W.; 卢, L.; 李, B.; 王, X.; 和戴, J. 2020. 可变形 DETR: 用于端到端目标检测的可变形 Transformer. *arXiv preprint arXiv:2010.04159*.